

Navigating in a shocking world

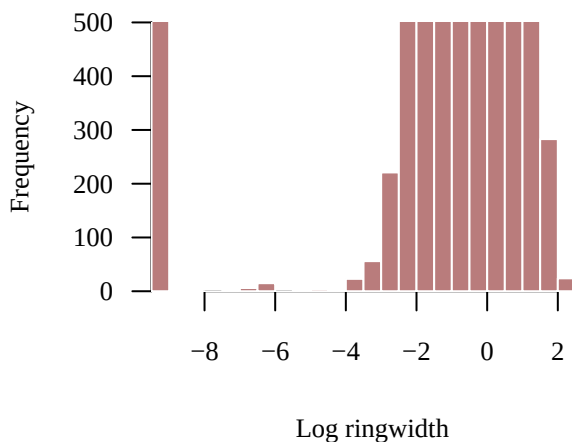
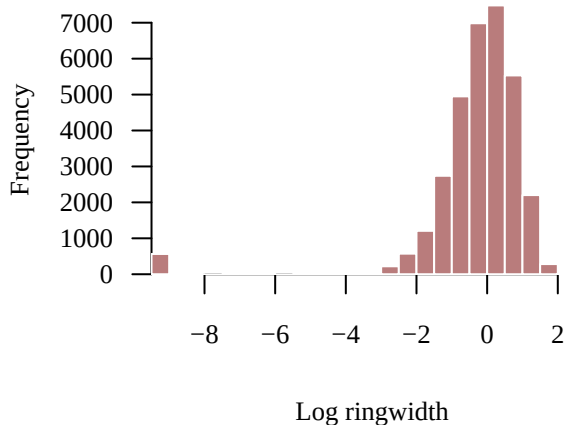
It's time to expand the dataset I'm using.

I look at all the *Pinus ponderosa* in Arizona, i.e. 914 trees on 20 stands (from 1980 onwards).

Look at the end of the document to see the Stan model.

The data

```
data <- readRDS(file = file.path(wd, 'output/model', 'data_15nov2025_azPIP0.rds'))
par(mfrow = c(1,2))
hist(data$log_rw_obs, col = util$c_mid, border = 'white', main = '',
      xlab = "Log ringwidth")
hist(data$log_rw_obs, col = util$c_mid, border = 'white', main = '',
      xlab = "Log ringwidth",
      ylim = c(0,500))
```



As a first guess, -4 seems like a good enough limit to define the (non-)centered configuration:

```
data$w_sck <- ifelse(data$log_rw_obs < -4, 1, 0)
```

A first fit

```

run <- FALSE
if(run){
  fit <- stan(file=file.path(wd, 'model', 'stan',
                             'model8_with3predictors_nopooling_standphi_nonconcordantshock.stan'),
             data=data, seed=5838293,
             chains = 4,
             warmup=1000, iter=2024, refresh=10)
  saveRDS(fit, file.path(wd, 'model/shocks/output/azPIPO', 'fit.rds'))
}else{
  fit <- readRDS(file.path(wd, 'model/shocks/output/azPIPO', 'fit.rds'))
}

diagnostics1 <- util$extract_hmc_diagnostics(fit)
util$check_all_hmc_diagnostics(diagnostics1)

```

Chain 1: 7 of 1024 transitions (0.68%)
saturated the maximum treedepth of 10.
Chain 1: E-FMI = 0.002.

Chain 2: 233 of 1024 transitions (22.75%)
saturated the maximum treedepth of 10.
Chain 2: E-FMI = 0.002.

Chain 3: 107 of 1024 transitions (10.45%)
saturated the maximum treedepth of 10.
Chain 3: E-FMI = 0.024.

Chain 4: 96 of 1024 transitions (9.38%)
saturated the maximum treedepth of 10.
Chain 4: E-FMI = 0.003.

Numerical trajectories that saturate the maximum treedepth have terminated prematurely. Increasing max_depth above 10 should result in more expensive, but more efficient, Hamiltonian transitions.

E-FMI below 0.2 arise when a funnel-like geometry obstructs how effectively Hamiltonian trajectories can explore the target distribution.

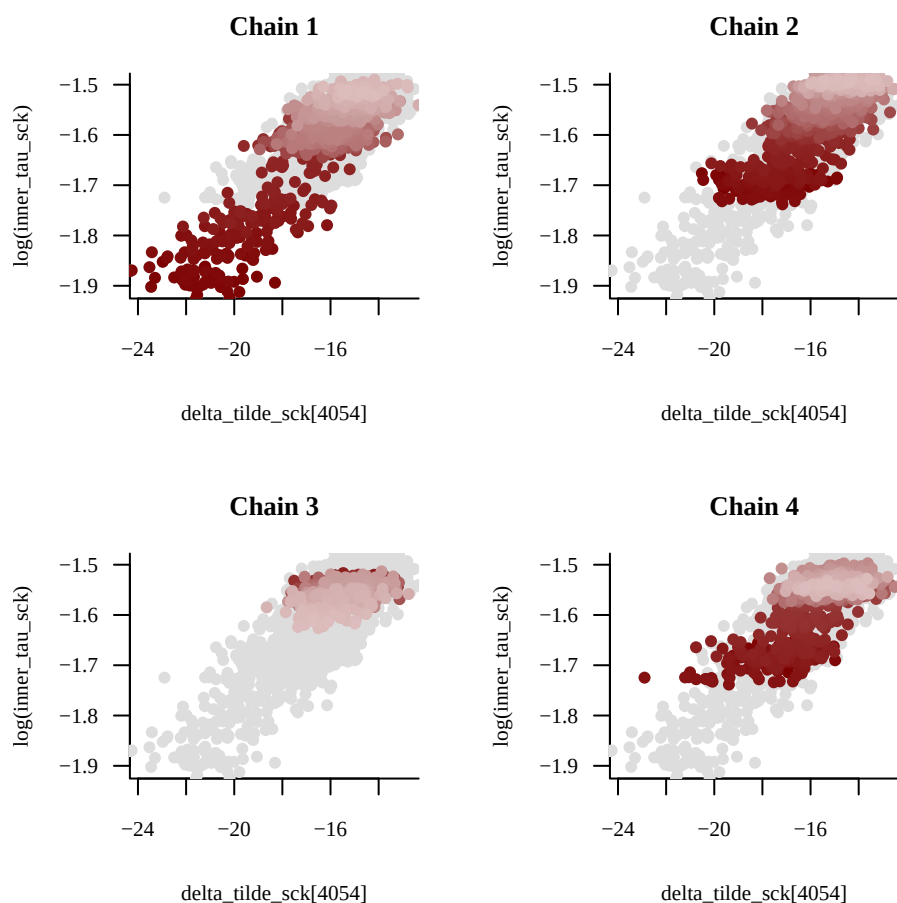
This first run is mostly to refine my (non-)centered configuration.

We can see that we're missing some shocks. For example, we can look at the first biggest negative shock that was not centered:

```

samples1 <- util$extract_expectand_vals(fit)
name_shocks <- paste("delta_tilde_sck[", 1:data$N, "]", sep='')
median_shocks <- sapply(name_shocks, function(x) util$ensemble_mcmc_quantile_est(samples1[[x]], 0.5))
last10mins <- sort(median_shocks[data$w_sck == 0])[1:10]
name_x <- sub("\\...*$", "", names(last10mins[1]))
util$plot_pairs_by_chain(samples1[[name_x]], name_x,
                        log(samples1[['inner_tau_sck']]), 'log(inner_tau_sck)')

```



To do this, I used the posterior median of the shocks and define an arbitrarily threshold:

```
par(mfrow = c(1,2))
hist(median_shocks, breaks = 50, col = util$c_mid, border = 'white', main = '')
hist(median_shocks, breaks = 50, col = util$c_mid, border = 'white', main = '',
      ylim = c(0,1000))
data$w_sck <- ifelse(abs(median_shocks) > 3, 1, 0)
```

A second fit

```
run <- FALSE
if(run){
  fit <- stan(file=file.path(wd, 'model', 'stan',
                             'model8_with3predictors_nopooling_standphi_nonconcordantshock.stan'),
             data=data, seed=5838293,
             chains = 4,
             warmup=1000, iter=2024, refresh=10)
  saveRDS(fit, file.path(wd, 'model/shocks/output/azPIP0', 'fit2.rds'))
}else{
  fit <- readRDS(file.path(wd, 'model/shocks/output/azPIP0', 'fit2.rds'))
}
```

Let's look at the diagnostics!

```
diagnostics2 <- util$extract_hmc_diagnostics(fit)
util$check_all_hmc_diagnostics(diagnostics2)
```

Chain 1: E-FMI = 0.017.

Chain 2: 4 of 1024 transitions (0.4%) diverged.

Chain 2: E-FMI = 0.003.

Chain 2: Average proxy acceptance statistic (0.715)
is smaller than 90% of the target (0.801).

Chain 3: E-FMI = 0.026.

Chain 4: E-FMI = 0.006.

Divergent Hamiltonian transitions result from unstable numerical trajectories. These instabilities are often due to degenerate target geometry, especially "pinches". If there are only a small number of divergences then running with `adept_delta` larger than 0.801 may reduce the instabilities at the cost of more expensive Hamiltonian transitions.

E-FMI below 0.2 arise when a funnel-like geometry obstructs how effectively Hamiltonian trajectories can explore the target distribution.

A small average proxy acceptance statistic indicates that the adaptation of the numerical integrator step size failed to converge. This is often due to discontinuous or imprecise gradients.

There are many signs that the posterior computation might be inaccurate.

At the parameters level (I excluded the stand-level GP and the tree-level shocks):

```
samples2 <- util$extract_expectand_vals(fit)
parameters <- names(samples2)
parameters <- parameters[!grepl("delta", parameters)]
parameters <- parameters[!grepl("f_", parameters)]
parameters <- parameters[!grepl("log_rw_pred", parameters)]
base_samples <- util$filter_expectands(samples2, parameters)
util$check_all_expectand_diagnostics(base_samples)
```

alpha:

Chain 2: $\hat{\text{ESS}}$ (52.052) is smaller than desired (100).

beta_gdd[1]:

Chain 1: $\hat{\text{ESS}}$ (92.683) is smaller than desired (100).

Chain 2: $\hat{\text{ESS}}$ (55.041) is smaller than desired (100).

beta_sm[1]:

Chain 1: $\hat{\text{ESS}}$ (85.700) is smaller than desired (100).

Chain 2: $\hat{\text{ESS}}$ (80.722) is smaller than desired (100).

Chain 4: $\hat{\text{ESS}}$ (76.048) is smaller than desired (100).

beta_vpd[1]:

Chain 1: $\hat{\text{ESS}}$ (77.991) is smaller than desired (100).

Chain 2: $\hat{\text{ESS}}$ (49.776) is smaller than desired (100).

gamma_sh:

Chain 2: $\hat{\text{ESS}}$ (64.856) is smaller than desired (100).

Chain 3: $\hat{\text{ESS}}$ (59.227) is smaller than desired (100).

rho_sp[1]:

Chain 1: $\hat{\text{ESS}}$ (36.642) is smaller than desired (100).

Chain 3: $\hat{\text{ESS}}$ (28.005) is smaller than desired (100).

inner_tau_sck:

Split $\hat{\text{ESS}}$ (2.474) exceeds 1.1.

Chain 1: $\hat{\text{ESS}}$ (7.623) is smaller than desired (100).

Chain 2: $\hat{\text{ESS}}$ (5.940) is smaller than desired (100).

Chain 3: $\hat{\text{ESS}}$ (10.016) is smaller than desired (100).

Chain 4: $\hat{\text{ESS}}$ (4.374) is smaller than desired (100).

phi_sck[18]:

Chain 1: $\hat{\text{ESS}}$ (90.319) is smaller than desired (100).

omega_nonconc_sck:

Chain 1: $\hat{\text{ESS}}$ (88.808) is smaller than desired (100).

Chain 3: $\hat{\text{ESS}}$ (55.775) is smaller than desired (100).

sigma:

Split $\hat{\text{ESS}}$ (2.853) exceeds 1.1.

Chain 1: $\hat{\text{ESS}}$ (8.641) is smaller than desired (100).

Chain 2: $\hat{\text{ESS}}$ (5.314) is smaller than desired (100).

Chain 3: $\hat{\text{ESS}}$ (9.544) is smaller than desired (100).

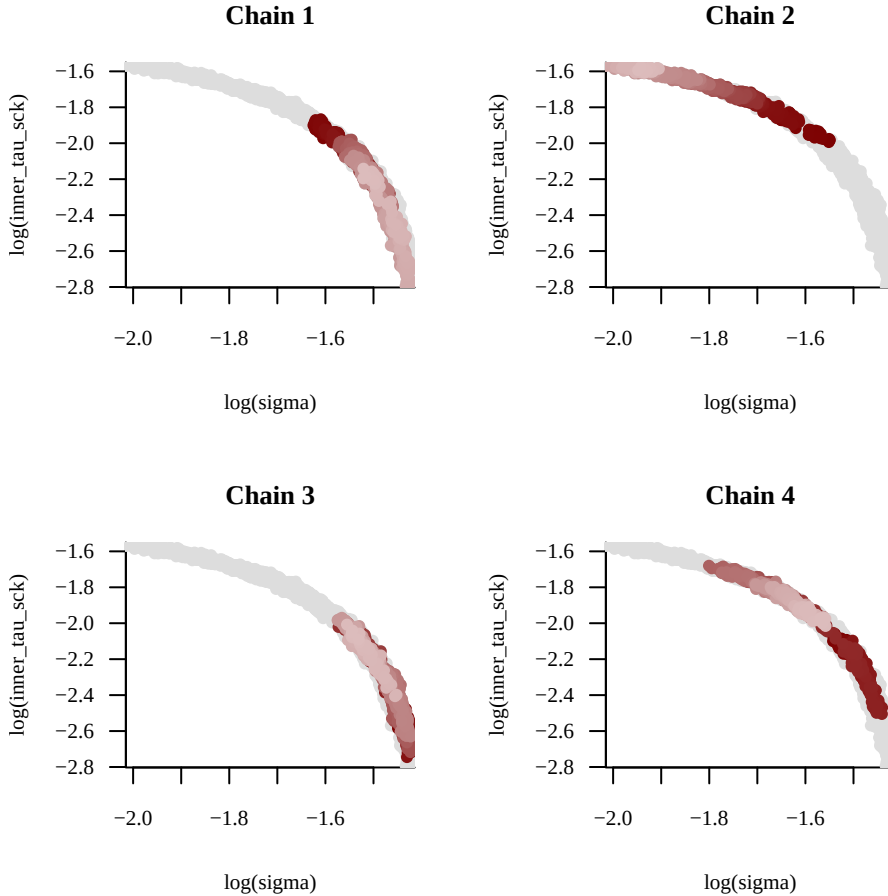
Chain 4: $\hat{\text{ESS}}$ (4.145) is smaller than desired (100).

Split Rhat larger than 1.1 suggests that at least one of the Markov chains has not reached an equilibrium.

Small empirical effective sample sizes result in imprecise Markov chain Monte Carlo estimators.

As a first step let's look at σ and $\tau_{\text{shock}}^{\text{inner}}$, because we expect a degeneracy there.

```
util$plot_pairs_by_chain(log(samples2[['sigma']]), 'log(sigma)',
                        log(samples2[['inner_tau_sck']]), 'log(inner_tau_sck)')
```

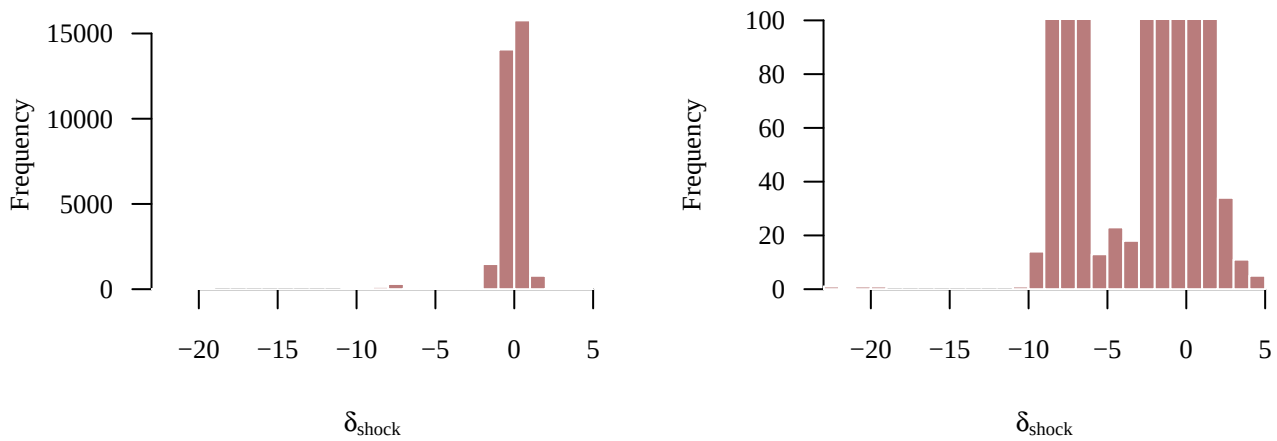


HMC has trouble exploring the geometry. Maybe because of some obstruction in exploring $\tau_{\text{shock}}^{\text{inner}}$ because of our crude initial (non-)centered configuration? To check this, let's look at some individual shocks.

Shock geometry

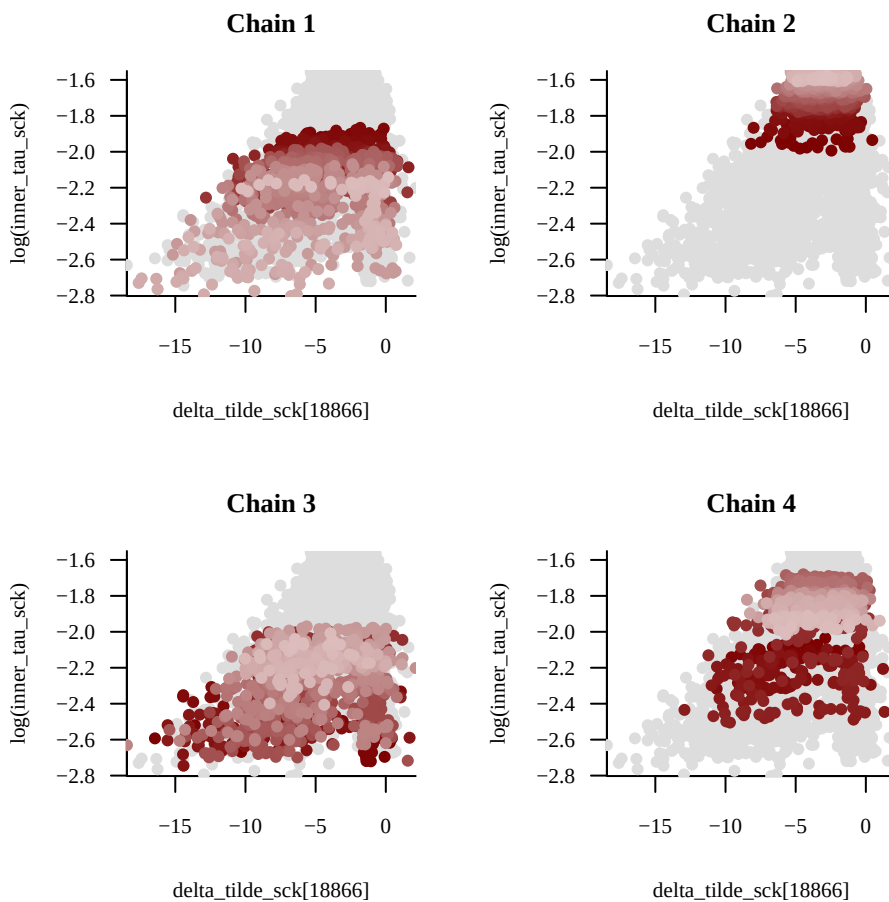
```
name_shocks <- paste("delta_tilde_sck[", 1:data$N, "]", sep='')
median_shocks <- sapply(name_shocks, function(x) util$ensemble_mcmc_quantile_est(samples2[[x]], 0.5))

par(mfrow = c(1,2))
hist(median_shocks, col = util$c_mid, border = 'white', main = '',
     xlab = expression(delta[shock]), breaks = 30)
hist(median_shocks, col = util$c_mid, border = 'white', main = '',
     xlab = expression(delta[shock]), breaks = 30,
     ylim = c(0,100))
```



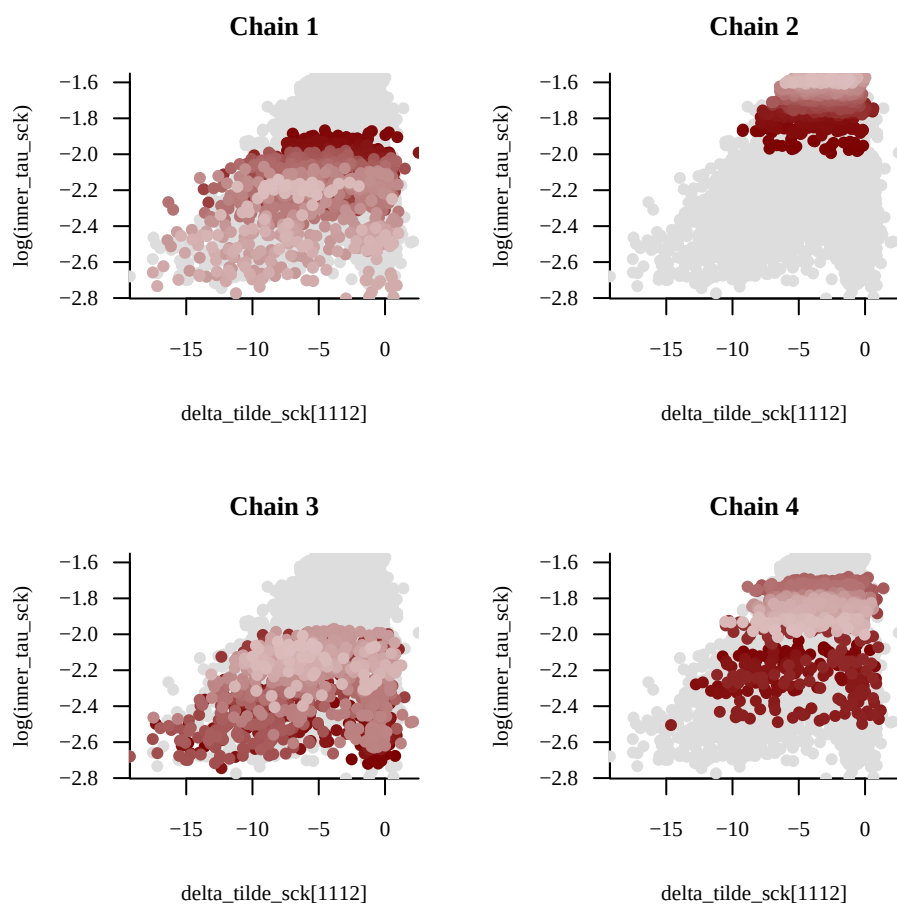
We can check for some evidence of big shocks that were not centered... For example, the first biggest negative shock that was not centered:

```
last10mins <- sort(median_shocks[data$w_sck == 0])[1:10]
name_x <- sub("\\..*$", "", names(last10mins[1]))
util$plot_pairs_by_chain(samples2[[name_x]], name_x,
  log(samples2[['inner_tau_sck']]), 'log(inner_tau_sck)')
```



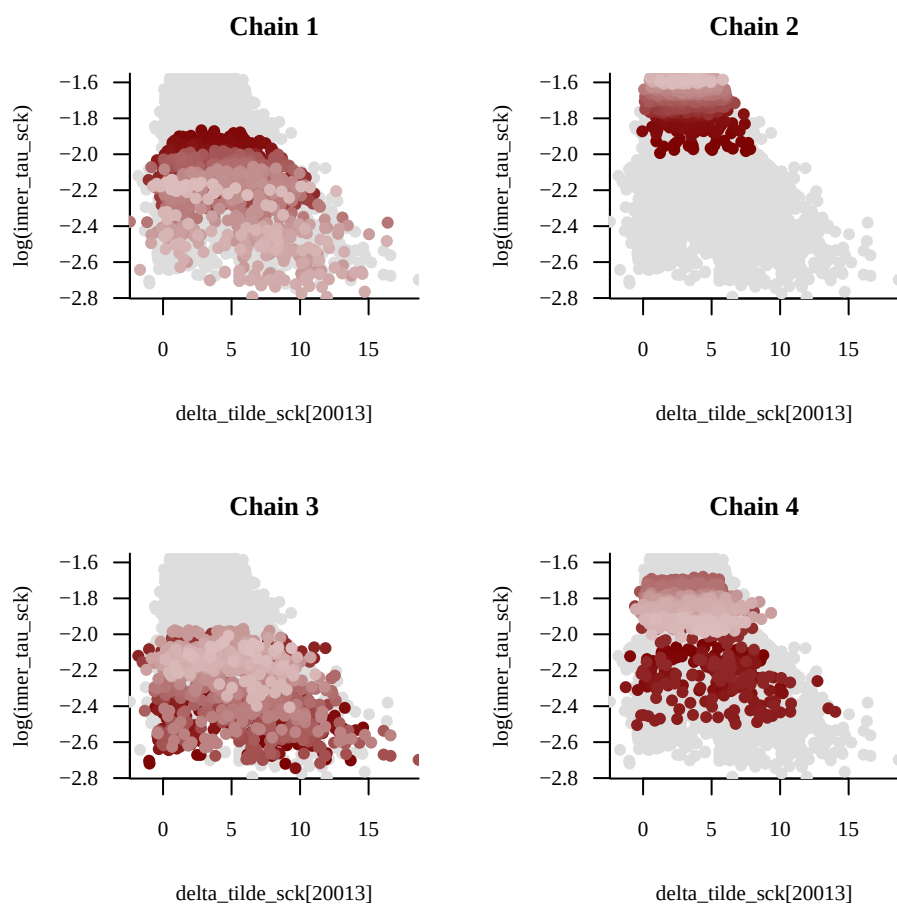
Or the second biggest negative shock that was not centered:

```
name_x <- sub("\\..*$", "", names(last10mins[2]))
util$plot_pairs_by_chain(samples2[[name_x]], name_x,
  log(samples2[['inner_tau_sck']]), 'log(inner_tau_sck)')
```



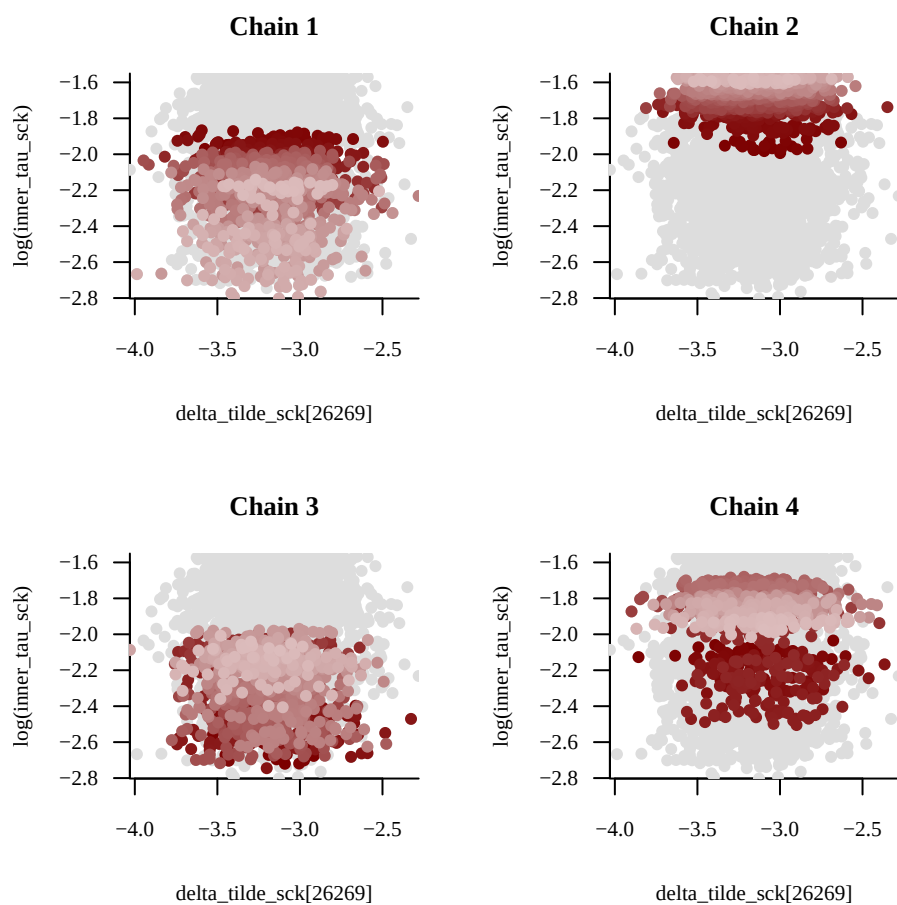
And for a big positive shock:

```
first10maxs <- sort(median_shocks[data$w_sck == 0], decreasing = TRUE)[1:10]
name_x <- sub("\\..*$", "", names(first10maxs[1]))
util$plot_pairs_by_chain(samples2[[name_x]], name_x,
  log(samples2[['inner_tau_sck']]), 'log(inner_tau_sck)')
```



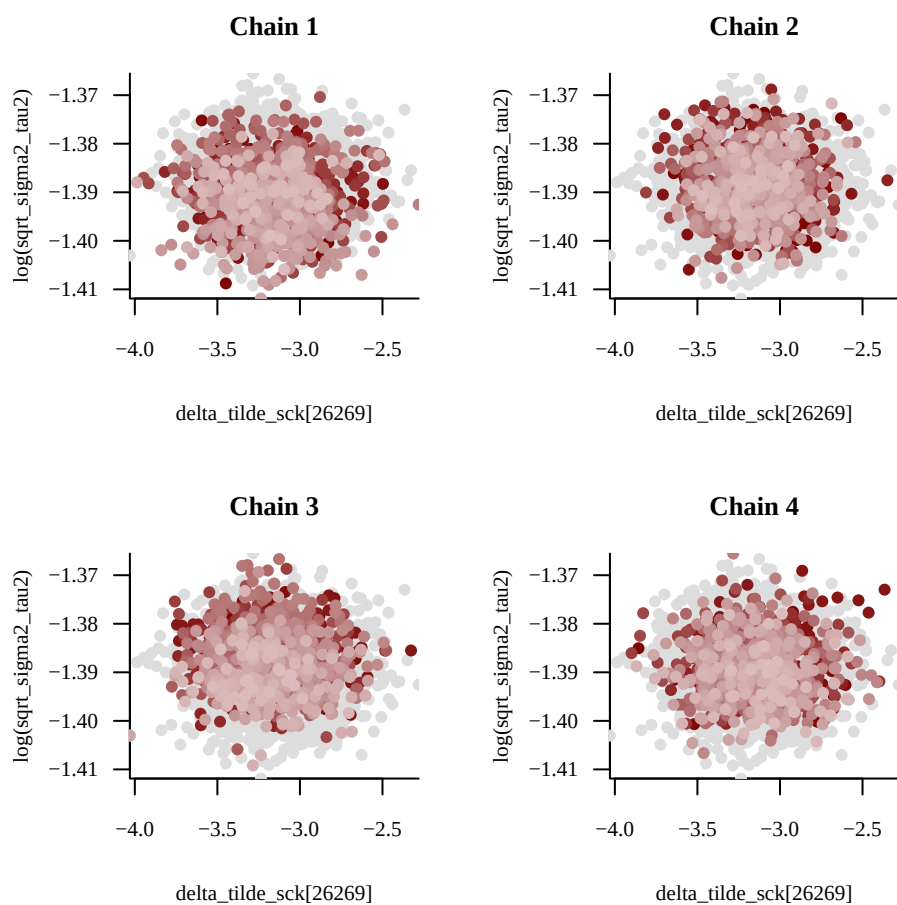
However, we don't see any evidence of non-shocks that we should not have centered:

```
absmins <- sort(abs(median_shocks[data$w_sck == 1]))[1:10]
name_x <- sub("\\..*$", "", names(absmins[1]))
util$plot_pairs_by_chain(samples2[[name_x]], name_x,
  log(samples2[['inner_tau_sck']]), 'log(inner_tau_sck)')
```



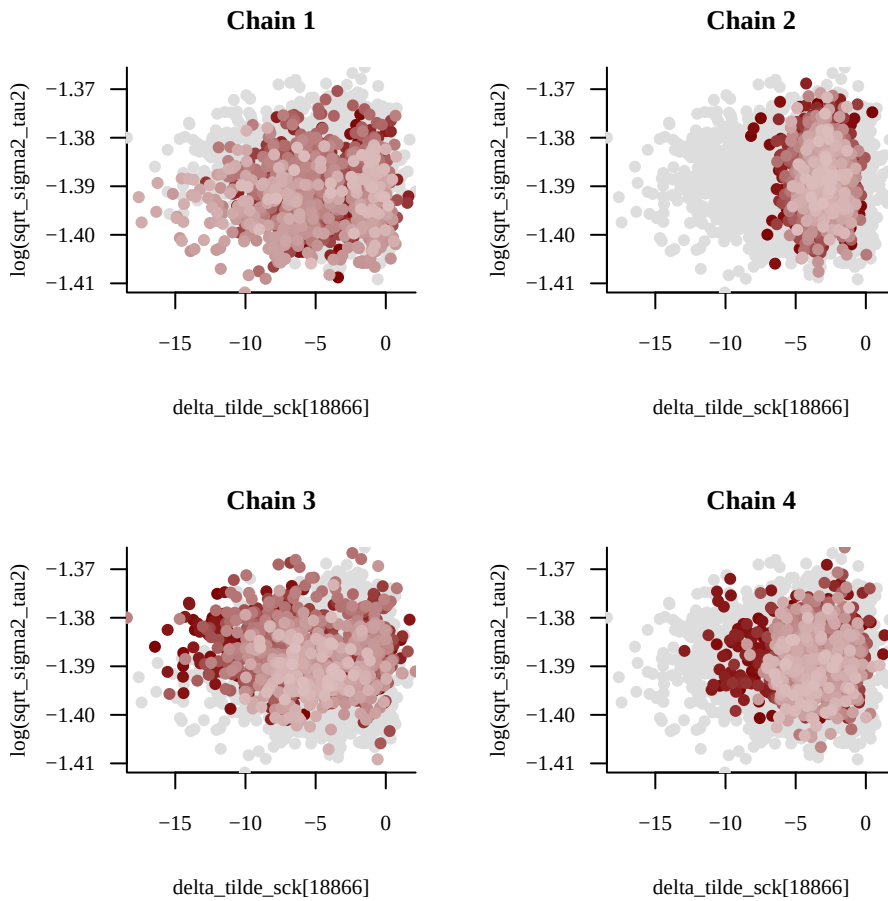
The spherical geometry is more obvious if we look at $\sqrt{\sigma^2 + \tau_{\text{shock}}^{\text{inner}}^2}$:

```
samples2[['sqrt_sigma2_tau2']] <- sqrt(samples2[['inner_tau_sck']]^2 + samples2[['sigma']]^2)
name_x <- sub("\\..*$", "", names(absmins[1]))
util$plot_pairs_by_chain(samples2[[name_x]], name_x,
  log(samples2[['sqrt_sigma2_tau2']]), 'log(sqrt_sigma2_tau2)')
```



But what if I look at $\sqrt{\sigma^2 + \tau_{\text{shock}}^{\text{inner}2}}$ for the biggest negative shock that was not centered?

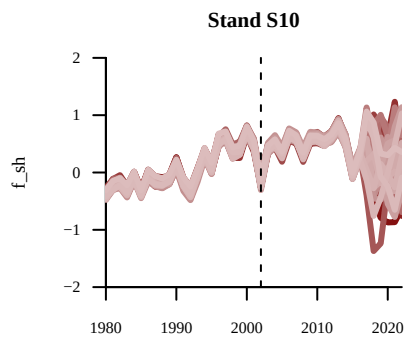
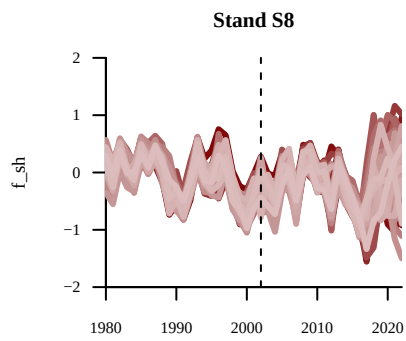
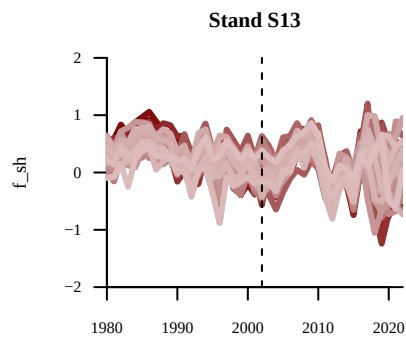
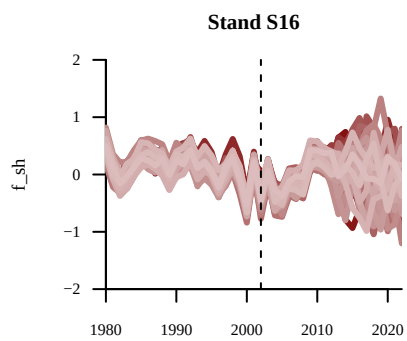
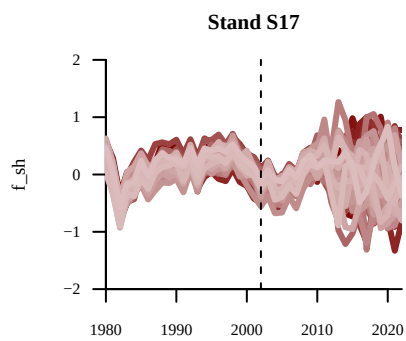
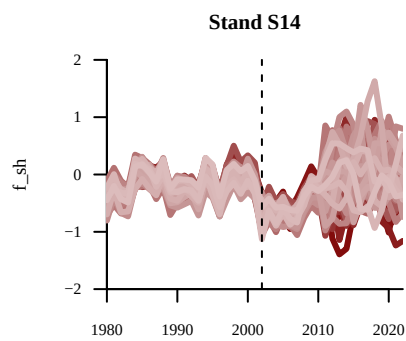
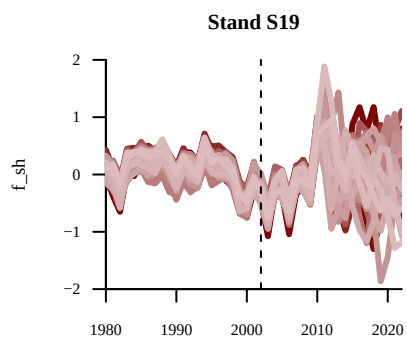
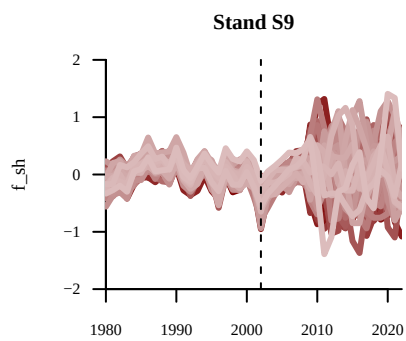
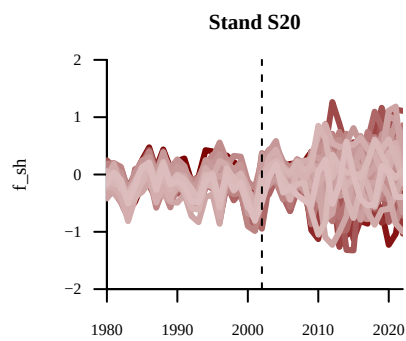
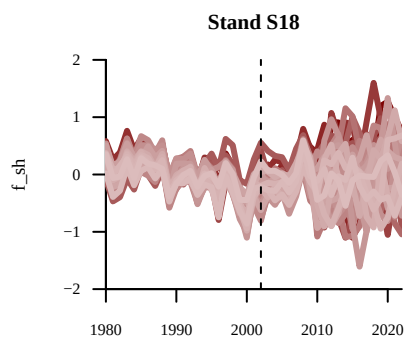
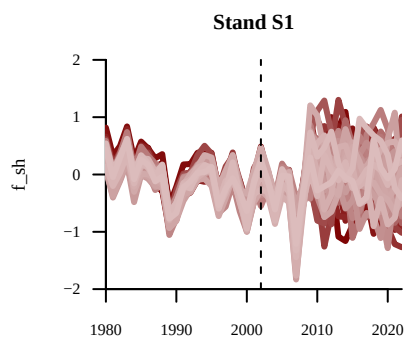
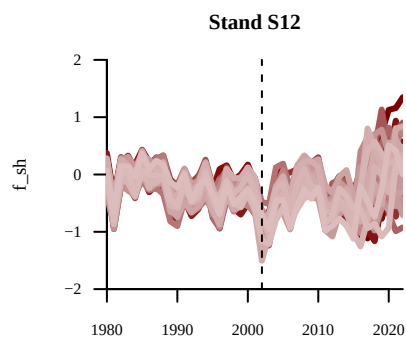
```
name_x <- sub("\\.\\.*$", "", names(last10mins[1]))
util$plot_pairs_by_chain(samples2[[name_x]], name_x,
  log(samples2[['sqrt_sigma2_tau2']]), 'log(sqrt_sigma2_tau2)')
```

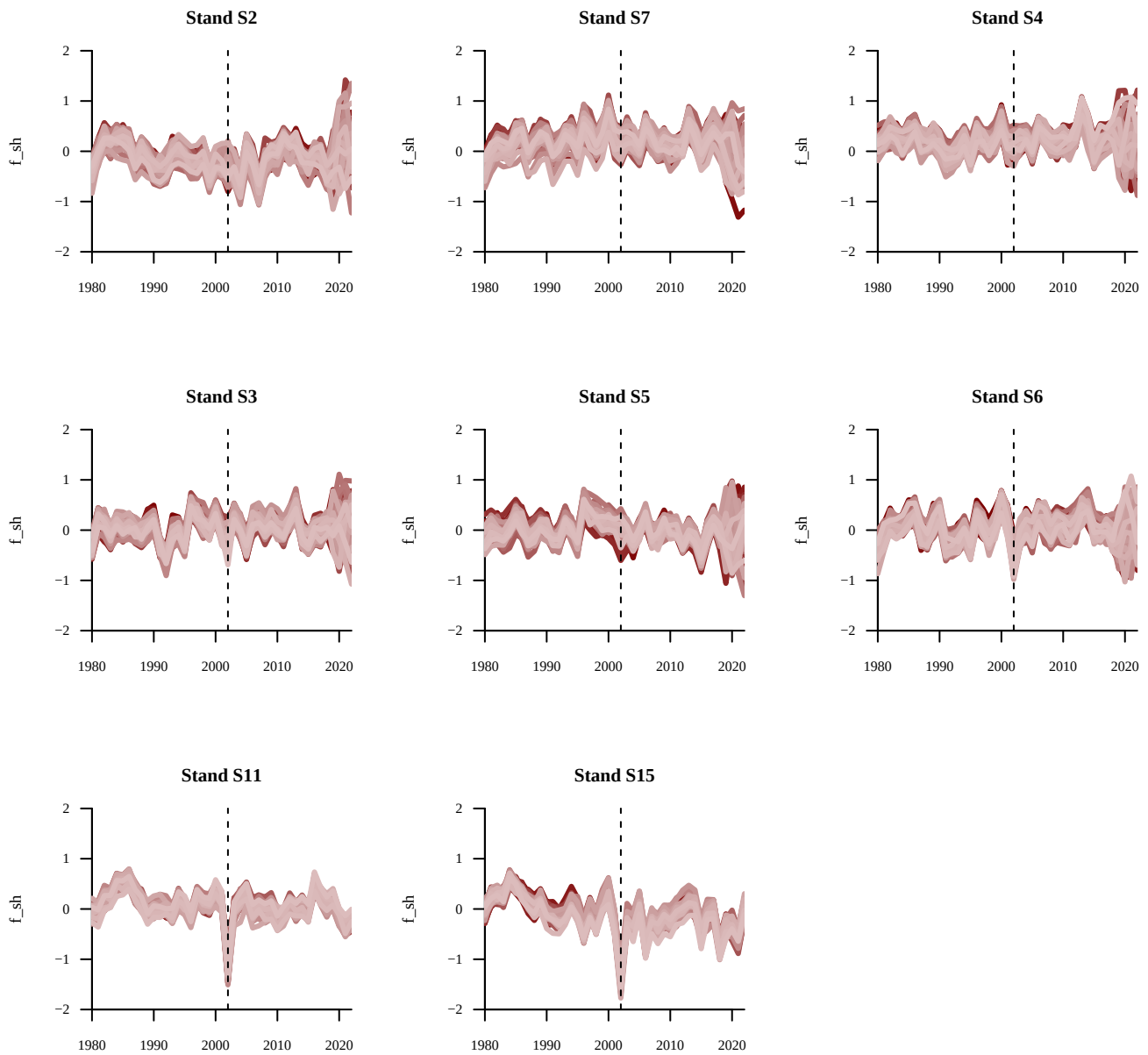


Comparing shocks across stands

A first approach to identify if our shock model is capturing lack of growth as we want is to look at the stand-level GPs. The dashed line corresponds to 2002, because we expect it to be a shock year.

```
par(mfrow = c(4,3))
for(s in 1:data$N_stands){
  names <- sapply(1:data$N_all_years,
    function(y) paste0('f_sh[', s, ',', y, ']'))
  util$plot_realizations(samples2, names, plot_xs = data$all_years[1:data$N_all_years], N_plots = 50,
    main = paste0('Stand ', data$uniq_stand_ids[s]), ylab = 'f_sh',
    display_xlim = data$all_years[c(1,data$N_all_years)], display_ylim = c(-2,2))
  abline(v = 2002, lty = 2)
}
```





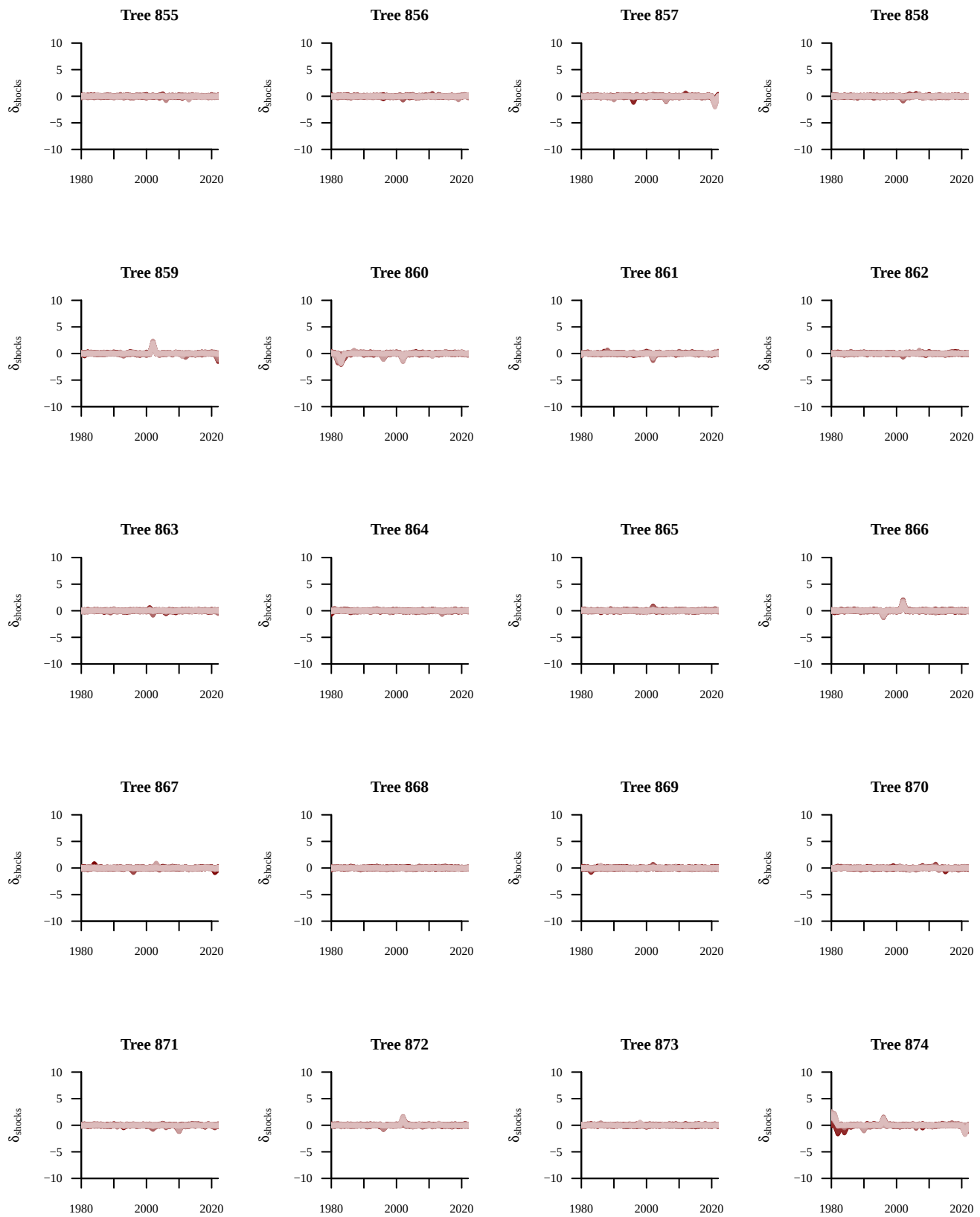
The stand-level GPs are still all very wiggly, but in particular we see evidence for some shocks in S11 and S15.

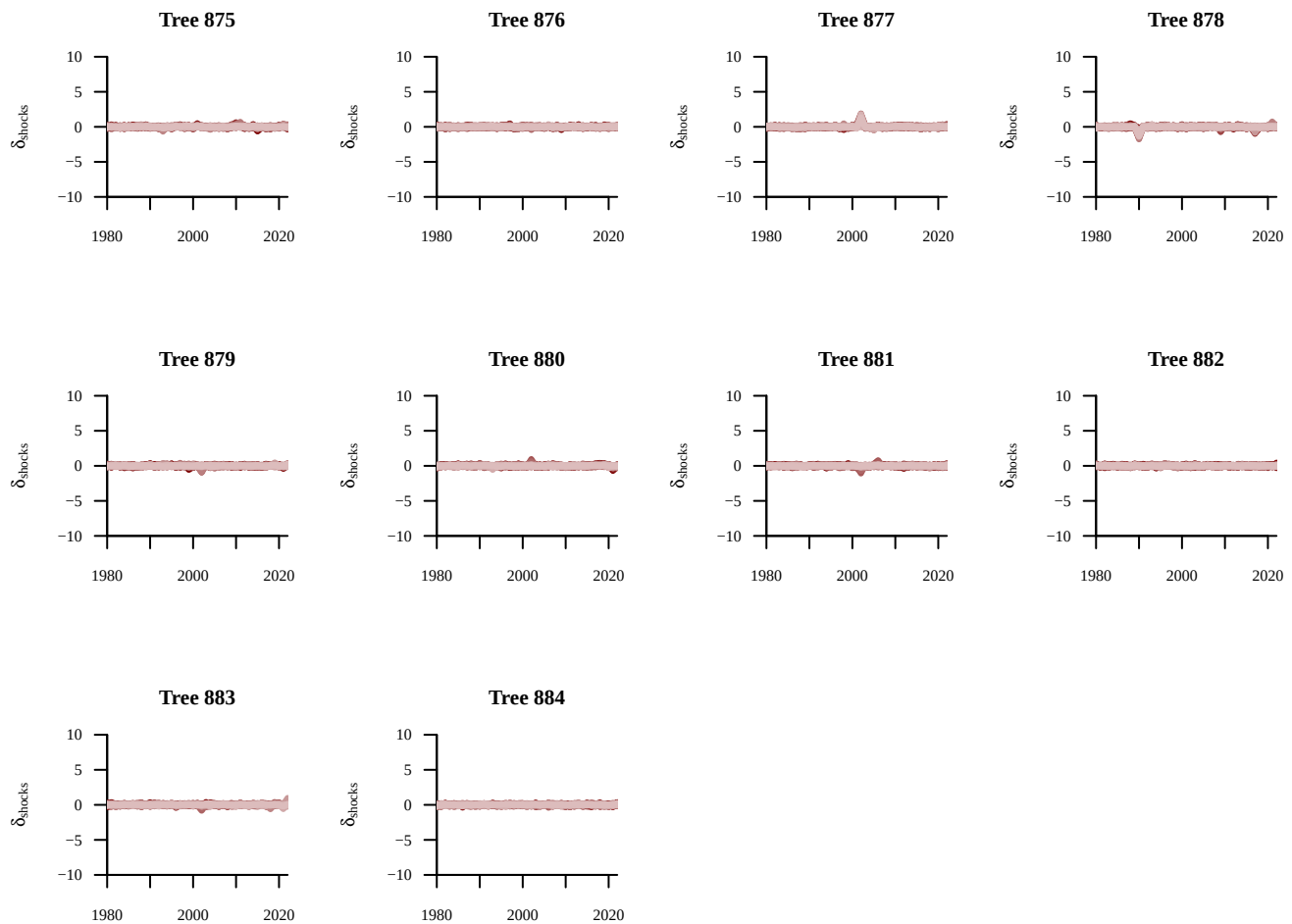
Let's look more closely at individual shocks for trees in S11.

```

trees_S11 <- which(data$stand_idx == which(data$uniq_stand_ids == 'S11'))
par(mfrow = c(5,4))
for(t in trees_S11){
  idxs_t <- data$tree_start_idx[t]:data$tree_end_idx[t]
  names <- sapply(idxs_t,
    function(x) paste0('delta_sck[', x, ']'))
  util$plot_realizations(samples2, names, plot_xs = data$years[idxs_t], N_plots = 100,
    ylab = expression(delta[shocks]),
    display_xlim = data$all_years[c(1,length(data$all_years))],
    display_ylim = c(-10,10),
    main = paste0('Tree ', t))
}

```





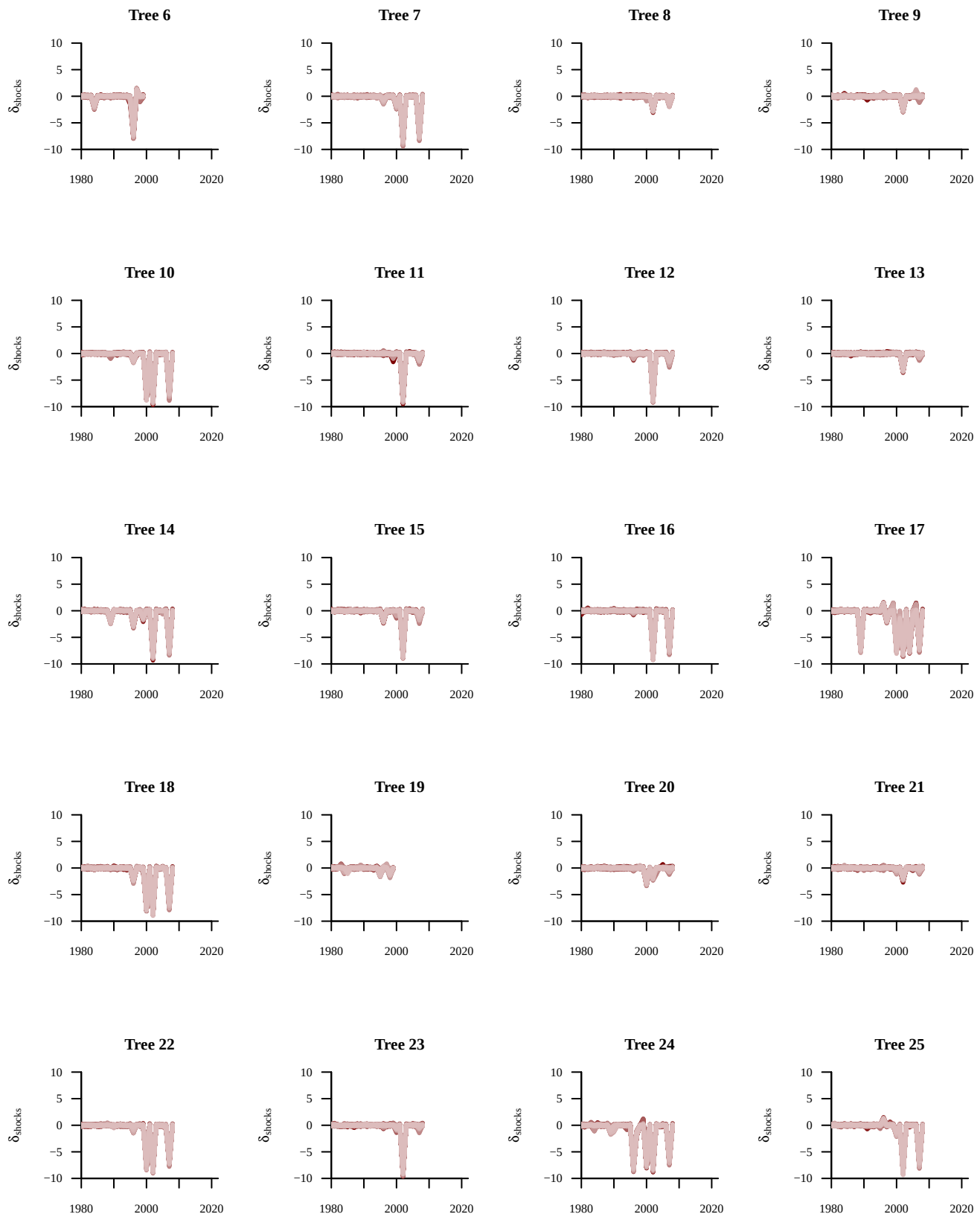
Mostly flat lines, i.e. our shock model does not capture anything.

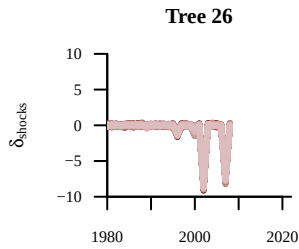
If we look at another stand (S1), we see evidence for a more coherent behavior with what we expect

```

trees_S1 <- which(data$stand_idx == which(data$uniq_stand_ids == 'S1'))
par(mfrow = c(5,4))
for(t in trees_S1){
  idxs_t <- data$tree_start_idx[t]:data$tree_end_idx[t]
  names <- sapply(idxs_t,
    function(x) paste0('delta_sck[', x, ']'))
  util$plot_realizations(samples2, names, plot_xs = data$years[idxs_t], N_plots = 100,
    ylab = expression(delta[shocks]),
    display_xlim = data$all_years[c(1,length(data$all_years))],
    display_ylim = c(-10,10),
    main = paste0('Tree ', t))
}

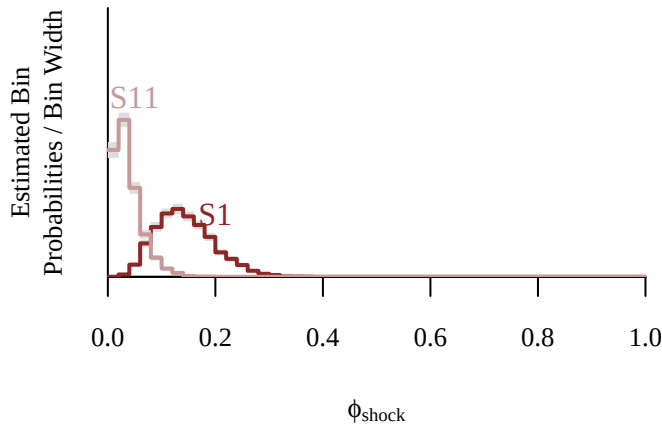
```





These two stands, S1 and S11, get a very different probability of shocks at the stand level.

```
par(mfrow = c(1,1))
util$plot_expectand_pushforward(samples2[[paste0('phi_sck[' , which(data$uniq_stand_ids == 'S1'), '']')]],
                               50, flim = c(0,1), ylim = c(0,30),
                               display_name=expression(phi[shock]))
text("S1", x = 0.2, y = 7, col = util$c_dark)
util$plot_expectand_pushforward(samples2[[paste0('phi_sck[' , which(data$uniq_stand_ids == 'S11'), '']')]],
                               50, flim = c(0,1), ylim = c(0,30),
                               display_name=expression(phi[shock]), add = TRUE,
                               col = util$c_light_highlight)
text("S11", x = 0.05, y = 20, col = util$c_light_highlight)
```



Do we obtain posteriors coherent with our domain expertise?

Despite the numerous issues with the posterior computation, let's take a quick look at the limited inference we get from the model.

In particular, let's look at the shock-related parameters (the plot for ϕ_{shock} is quite messy because each plot gets a different ϕ_{shock}).

```
# Quickly look at posteriors
nf <- layout(
  matrix(c(1,1,1,2,2,2,
           3,3,4,4,5,5),
         ncol=6, byrow=TRUE)
)
util$plot_expectand_pushforward(samples2[['inner_tau_sck']], 50,
                               flim = c(0,1.5),
                               display_name=expression(tau[shock]^inner))

xs <- seq(0, 2, 0.01)
ys <- dnorm(xs, 0, log(1.2) / 2.57)
lines(xs, ys, lwd=2, col=util$c_mid_teal)

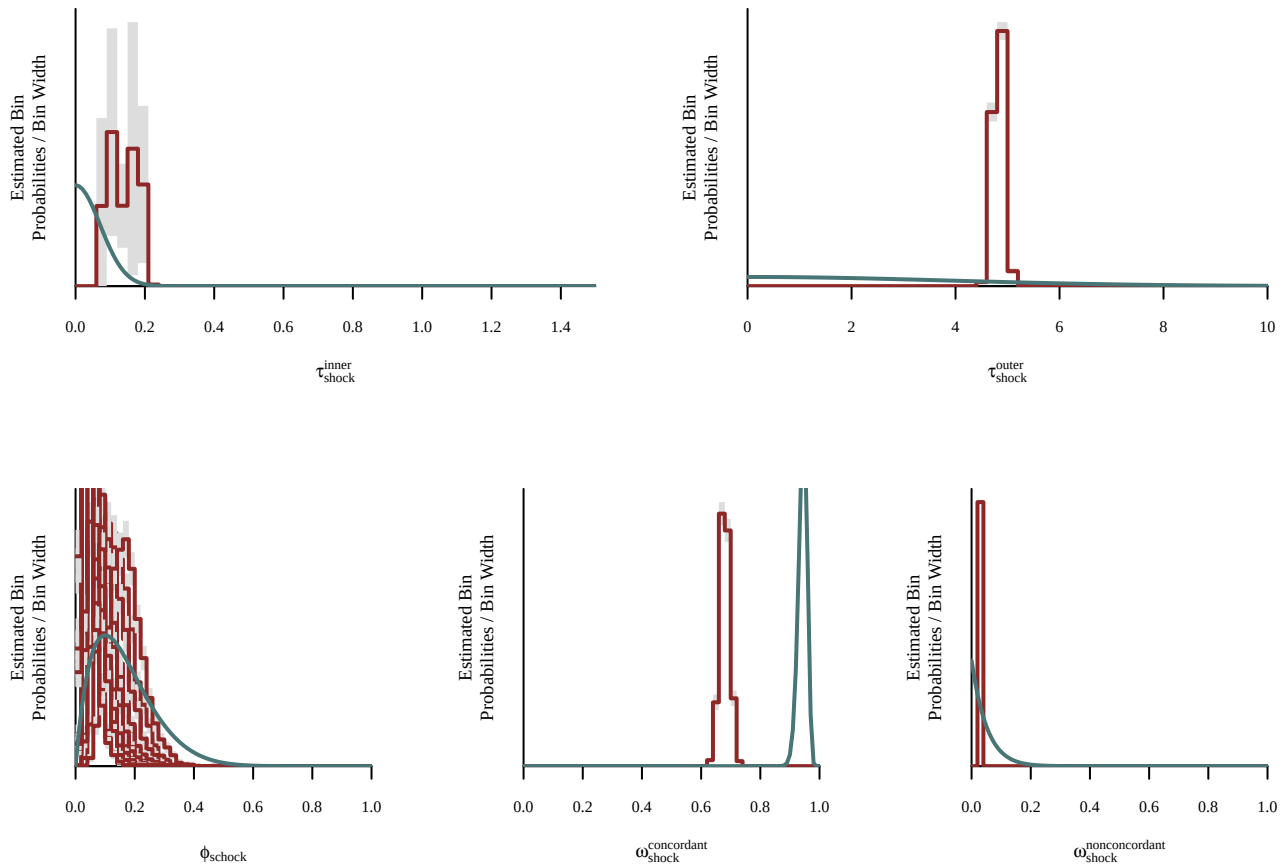
util$plot_expectand_pushforward(samples2[['outer_tau_sck']], 50,
                               flim = c(0,10),
                               display_name=expression(tau[shock]^outer))

xs <- seq(0, 10, 0.01)
ys <- dnorm(xs, 0, 10 / 2.57)
lines(xs, ys, lwd=2, col=util$c_mid_teal)
```

```

util$plot_expectand_pushforward(samples2[['phi_sck[1]']], 50,
                                flim = c(0,1),
                                display_name=expression(phi[schock]))
for(s in 2:data$N_stands){
  util$plot_expectand_pushforward(samples2[[paste0('phi_sck[' ,s,']')]]], 50,
                                flim = c(0,1),
                                display_name=expression(phi[schock]), add = T)
}
xs <- seq(0, 1, 0.01)
ys <- dbeta(xs, 2, 10)
lines(xs, ys, lwd=2, col=util$c_mid_teal)
util$plot_expectand_pushforward(samples2[['omega_conc_sck']], 50,
                                flim = c(0,1),
                                display_name=expression(omega[schock]^concordant))
xs <- seq(0, 1, 0.01)
ys <- dbeta(xs, 230, 14)
lines(xs, ys, lwd=2, col=util$c_mid_teal)
util$plot_expectand_pushforward(samples2[['omega_nonconc_sck']], 50,
                                flim = c(0,1),
                                display_name=expression(omega[schock]^nonconcordant))
xs <- seq(0, 1, 0.01)
ys <- dbeta(xs, 1, 20)
lines(xs, ys, lwd=2, col=util$c_mid_teal)

```



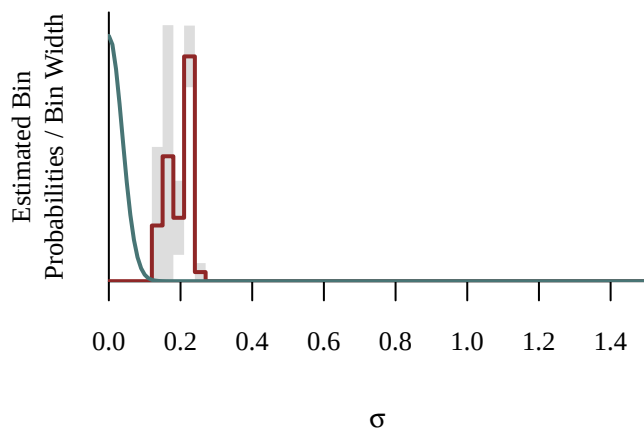
It's quite clear that the behavior of $\omega_{\text{shock}}^{\text{concordant}}$, i.e. $p(z_{\text{tree shock}} = 1 | z_{\text{stand shock}} = 1)$, is not coherent with what I was expecting.

Also, let's look at posterior distribution for σ :

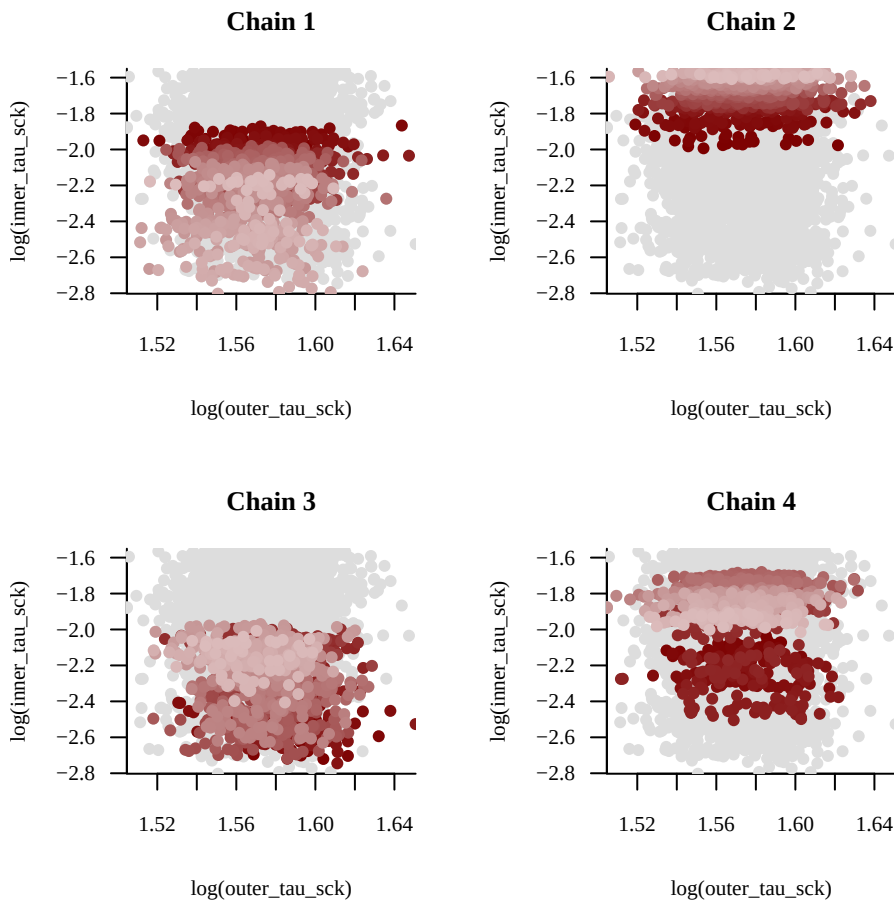
```

# Quickly look at posteriors
par(mfrow=c(1,1))
util$plot_expectand_pushforward(samples2[['sigma']], 50,
                                flim = c(0,1.5),
                                display_name=expression(sigma))
xs <- seq(0, 2, 0.01)
ys <- dnorm(xs, 0, log(1.1) / 2.57)
lines(xs, ys, lwd=2, col=util$c_mid_teal)

```



```
util$plot_pairs_by_chain(log(samples2[['outer_tau_sck']]), 'log(outer_tau_sck)',
                        log(samples2[['inner_tau_sck']]), 'log(inner_tau_sck)')
```



The model

```
# we skip the function block
# many stuff commented out because I'm looking at only one species...
writeLines(
  readLines(file.path(wd, 'model', 'stan',
    'model18_with3predictors_nopooling_standphi_nonconcordantshock.stan'))[58:340])
```

```

data {
  int<lower=1> N;           // Total number of observations
  int<lower=1> N_species;   // Number of species
  int<lower=1> N_trees;     // Number of trees
  int<lower=1> N_all_years; // Max number of years
  int<lower=1> N_stands;    // Number of stands
  // int<lower=1> N_clades;  // Number of clades - for now, Gymnosperms=1 or Angiosperms=2

  array[N_stands] int<lower=1, upper=N> N_stand_trees; // Number of trees per stand
  array[N_stands] int<lower=1, upper=N> N_stand_years; // Max. number of observed years per stand
  array[N_stands] int<lower=1, upper=N> stand_start_years_idx; // Indices of first year observed per stand

  // Indices of tree stands
  array[N_trees] int<lower=1, upper=N_stands> stand_idx;

  // Indices of tree species
  array[N_trees] int<lower=1, upper=N_species> species_idx;

  // Number of observations per tree
  array[N_trees] int<lower=1, upper=N> N_years;

  // Years spanned by all observations
  array[N_all_years] real all_years;

  // Tree observation years
  array[N] real years;

  // Indices of tree observation years
  array[N] int<lower=1, upper=N_all_years> all_years_idx;

  // Ragged array indexing for trees
  array[N_trees] int<lower=1, upper=N> tree_start_idx;
  array[N_trees] int<lower=1, upper=N> tree_end_idx;

  // Ragged array indexing for stands
  array[N_stands] int<lower=1, upper=N> stand_trees_start_idx;
  array[N_stands] int<lower=1, upper=N> stand_trees_end_idx;

  vector[N] log_rw_obs; // Log of Observed Ring Width Per 1 mm
  vector[N] gdd_obs; // Observed gdd (all year) (x100 degC)
  vector[N] sm_obs; // Observed soil moisture (MJJ) (%)
  vector[N] vpd_obs; // Observed VPD (JMJJA) (hPa)

  // Indices of species clades (Gymnosperms=1 or Angiosperms=2)
  // array[N_species] int<lower=1, upper=N_clades> clade_idx;

  // Partial centering parameters for growth shocks
  vector<lower=0, upper=1>[N] w_sck;
}

transformed data {
  real gdd0 = 10;
  real sm0 = 25;
  real vpd0 = 8;
}

parameters {
  real alpha; // Log ring width baseline

  // GDD slope (1/100degC)
  // vector[N_clades] mu_gdd;
  // vector<lower=0>[N_clades] tau_gdd;
  vector[N_species] beta_gdd;

  // SM slope (1/%)
  // vector[N_clades] mu_sm;
  // vector<lower=0>[N_clades] tau_sm;
  vector[N_species] beta_sm;

  // VPD slope (1/hPa)
  // vector[N_clades] mu_vpd;
  // vector<lower=0>[N_clades] tau_vpd;
  vector[N_species] beta_vpd;

  // Short-term proportional growth functional behavior
  array[N_stands] vector[N_all_years] f_tilde_sh; // Non-centered functional behavior
  real<lower=1> rho_sh; // Time scale
  real<lower=0> gamma_sh; // Marginal variation - now fixed to 1! (and scaled by kappa)

  // Lifetime proportional growth scale (here I implement the hard constraint on both clade and species parameters?)
  // vector<lower=rho_sh>[N_clades] mu_rho;
  // vector<lower=0>[N_clades] tau_rho;
  vector<lower=rho_sh>[N_species] rho_sp;

  // Lifetime proportional growth variation
  // vector<lower=0>[N_clades] mu_gamma;
  // vector<lower=0>[N_clades] tau_gamma;

```



```

vector<lower=0>[N_species] gamma_sp;
// Short-term proportional growth species scaling
// vector<lower=0>[N_clades] mu_kappa;
// vector<lower=0>[N_clades] tau_kappa;
// vector<lower=0>[N_species] kappa_sh;

// Growth shocks
vector[N] delta_tilde_sck; // Latent parameter for yearly shocks
real<lower=0> inner_tau_sck; // Inner yearly log variation scale
real<lower=inner_tau_sck> outer_tau_sck; // Outer yearly log variation scale (the shocks!)

// Probability of shocks
vector<lower=0, upper=1>[N_stands] phi_sck; // Probability of stand-level shock
real<lower=0, upper=1> omega_conc_sck; // Probability of tree-level shock given stand in shock (concordant shock)
real<lower=0, upper=omega_conc_sck> omega_nonconc_sck; // Probability of tree-level shock given stand NOT in shock (nonconcordant shock)

// Growth shock species scaling
// vector<lower=0>[N_clades] mu_kappa_sck;
// vector<lower=0>[N_clades] tau_kappa_sck;
// vector<lower=0>[N_species] kappa_sck;

// Proportional measurement error
real<lower=0> sigma;
}

transformed parameters {
  // array[N_species] real kappa_sh = append_array({1}, kappa_sh_free);

  array[N_stands] vector[N_all_years] f_sh;
  {
    matrix[N_all_years, N_all_years] cov
      = gp_exp_quad_cov(all_years, gamma_sh, rho_sh)
      + diag_matrix(rep_vector(1e-10, N_all_years));
    matrix[N_all_years, N_all_years] L_cov = cholesky_decompose(cov);

    for (s in 1:N_stands) {
      f_sh[s] = L_cov * f_tilde_sh[s];
    }
  }

  // vector[N_species] beta_gdd;
  // vector[N_species] beta_sm;
  // vector[N_species] beta_vpd;
  //
  // for (sp in 1:N_species) {
  //   beta_gdd[sp] = mu_gdd[clade_idx[sp]] + tau_gdd[clade_idx[sp]] * beta_gdd_tilde[sp];
  //   beta_sm[sp] = mu_sm[clade_idx[sp]] + tau_sm[clade_idx[sp]] * beta_sm_tilde[sp];
  //   beta_vpd[sp] = mu_vpd[clade_idx[sp]] + tau_vpd[clade_idx[sp]] * beta_vpd_tilde[sp];
  // }

  vector[N] delta_sck;
  for (i in 1:N){
    delta_sck[i] = pow(inner_tau_sck, 1 - w_sck[i]) * delta_tilde_sck[i];
  }
}

model {
  array[N_species] matrix[N_all_years, N_all_years] L_cov;
  for (sp in 1:N_species) {
    matrix[N_all_years, N_all_years] cov
      = gp_exp_quad_cov(all_years, gamma_sp[sp], rho_sp[sp])
      + diag_matrix(rep_vector(square(sigma), N_all_years));
    L_cov[sp] = cholesky_decompose(cov);
  }

  alpha ~ normal(0, 0.69); // 0.2 mm <~ exp(alpha) * 1 mm <~ 5 mm

  beta_gdd ~ normal(0, log(1.8) / 2.57); // -log(1.8) <~ beta_gsl <~ log(1.8)
  beta_sm ~ normal(0, log(1.8) / 2.57); // -log(1.8) <~ beta_gsl <~ log(1.8)
  beta_vpd ~ normal(0, log(1.8) / 2.57); // -log(1.8) <~ beta_gsl <~ log(1.8)

  rho_sp ~ lognormal(2.65, 0.135); // 10 <~ rho <~ 20

  gamma_sp ~ normal(0, log(10) / 2.57); // 0 <~ gamma <~ log(10)

  // kappa_sh ~ lognormal(0, 0.41 / 2.32); // 2/3 <~ kappa_sh <~ 3/2
  // kappa_sck ~ lognormal(0, 0.41 / 2.32); // 2/3 <~ kappa_sh <~ 3/2

  for (s in 1:N_stands)
    f_tilde_sh[s] ~ normal(0, 1);
  rho_sh ~ lognormal(1.7, 0.26); // 3 <~ rho_sh <~ 10
  gamma_sh ~ normal(0, log(3) / 2.57); // 0 <~ gamma_sh <~ log(3)

  // Shocks!
  for (s in 1:N_stands) {

```

```

array[N_stand_trees[s]] int stand_trees_idx = linspace_int_array(N_stand_trees[s],
  stand_trees_start_idx[s], stand_trees_end_idx[s]);
vector[N_stand_years[s]] log_p0 = rep_vector(0, N_stand_years[s]);
vector[N_stand_years[s]] log_p1 = rep_vector(0, N_stand_years[s]);

for(ts in 1:N_stand_trees[s]){
  int t = stand_trees_idx[ts];
  array[N_years[t]] int tree_idx = linspace_int_array(N_years[t], tree_start_idx[t], tree_end_idx[t]);
  array[N_years[t]] int all_years_idx_tree = all_years_idx[tree_idx];

  for(y in 1:N_years[t]) {
    int ys = all_years_idx_tree[y]-stand_start_years_idx[s]+1;

    log_p0[ys] += log_mix(omega_nonconc_sck,
      normal_lpdf(delta_tilde_sck[tree_idx[y]] | 0, pow(inner_tau_sck, w_sck[tree_idx[y]] - 1) * outer_tau_sck,
      normal_lpdf(delta_tilde_sck[tree_idx[y]] | 0, pow(inner_tau_sck, w_sck[tree_idx[y]])));
    log_p1[ys] += log_mix(omega_conc_sck,
      normal_lpdf(delta_tilde_sck[tree_idx[y]] | 0, pow(inner_tau_sck, w_sck[tree_idx[y]] - 1) * outer_tau_sck,
      normal_lpdf(delta_tilde_sck[tree_idx[y]] | 0, pow(inner_tau_sck, w_sck[tree_idx[y]])));
  }
}

for(y in 1:N_stand_years[s]) {
  target += log_mix(phi_sck[s], log_p1[y], log_p0[y]);
}
}

inner_tau_sck ~ normal(0, log(1.2) / 2.57);
outer_tau_sck ~ normal(0, 10 / 2.57);
phi_sck ~ beta(2, 10);
omega_conc_sck ~ beta(230, 14); // 0.9 <~ omega_conc_sck <~ 0.97 (most trees, but not ALL trees)
omega_nonconc_sck ~ beta(1, 20); // 0 <~ omega_conc_sck <~ 0.15 (should be rare, but... who knows?)

sigma ~ normal(0, log(1.1) / 2.57); // 0 <~ sigma <~ +log(1.1)
// sigma ~ gamma(29.5, 458.8); // log(1.05) <~ sigma <~ log(1.1)

for (t in 1:N_trees) {
  array[N_years[t]] int tree_idx
  = linspace_int_array(N_years[t],
    tree_start_idx[t],
    tree_end_idx[t]);

  array[N_years[t]] int all_years_idx_tree = all_years_idx[tree_idx];
  array[N_years[t]] real years_tree = years[tree_idx];

  int stand_idx = stand_idx[t];
  int species_idx = species_idx[t];

  // L_cov needs to be marginalized to the specific
  // observation years for each tree.
  matrix[N_years[t], N_years[t]] L_cov_tree = block(L_cov[species_idx], 1, 1, N_years[t], N_years[t]);

  vector[N_years[t]] gdd_obs_tree = gdd_obs[tree_idx];
  vector[N_years[t]] sm_obs_tree = sm_obs[tree_idx];
  vector[N_years[t]] vpd_obs_tree = vpd_obs[tree_idx];

  vector[N_years[t]] delta_sck_tree = delta_sck[tree_idx];

  vector[N_years[t]] mu = alpha
  + beta_gdd[species_idx] * (gdd_obs_tree - gdd0)
  + beta_sm[species_idx] * (sm_obs_tree - sm0)
  + beta_vpd[species_idx] * (vpd_obs_tree - vpd0)
  + f_sh[stand_idx, all_years_idx_tree]
  + delta_sck_tree;

  log_rw_obs[tree_idx] ~ multi_normal_cholesky(mu, L_cov_tree);
}
}

generated quantities {
  array[N] real log_rw_pred;

  for (t in 1:N_trees) {
    array[N_years[t]] int tree_idx = linspace_int_array(N_years[t], tree_start_idx[t], tree_end_idx[t]);
    array[N_years[t]] real years_tree = years[tree_idx];
    array[N_years[t]] int all_years_idx_tree = all_years_idx[tree_idx];

    int stand_idx = stand_idx[t];
    int species_idx = species_idx[t];

    vector[N_years[t]] gdd_obs_tree = gdd_obs[tree_idx];
    vector[N_years[t]] sm_obs_tree = sm_obs[tree_idx];
    vector[N_years[t]] vpd_obs_tree = vpd_obs[tree_idx];

    vector[N_years[t]] delta_sck_tree = delta_sck[tree_idx];

    vector[N_years[t]] mu1 = alpha

```

```

+ beta_gdd[species_idx] * (gdd_obs_tree - gdd0)
+ beta_sm[species_idx] * (sm_obs_tree - sm0)
+ beta_vpd[species_idx] * (vpd_obs_tree - vpd0)
+ f_sh[stand_idx, all_years_idx]_tree]
+ delta_sck_tree;

vector[N_years[t]] log_rw = gp_pred_rng(years_tree, log_rw_obs[tree_idx], years_tree,
                                         mu1, gamma_sp[species_idx], rho_sp[species_idx], sigma, 1e-10);

// rw[tree_idx] = exp(log_rw[tree_idx]);
log_rw_pred[tree_idx] = normal_rng(log_rw, sigma);
}
}

```